City Ground Zero

How the smart-city simulation works: design, architecture and operation

8 × 8	200	~200	566	11
districts, 81 junctions	citizens in 80 households	simulated vehicles (SUMO)	sensors and actuators	incident scenarios

1. What it is

City Ground Zero is a live, three-dimensional smart-city simulator. A city of roads, buildings, parks and street furniture is populated by traffic, emergency services and a society of citizens who work, shop, take breaks and go home. The city is instrumented like a real smart city: hundreds of sensors report what is happening and actuators (signals, switches, valves, lights) can change it. An operator watches and controls everything from a single control-room window, and the same sensors and actuators are open to software through a standard interface (MCP), so an external program or an AI assistant can monitor and run the city.

The guiding idea is to **reuse proven simulators for what they do best** and to keep one clear owner for every piece of state. Traffic physics belongs to a professional traffic simulator, the game engine draws the world and hosts the city systems, and every other part (citizens, incidents, utilities, sensors) reads from and writes to those owners rather than keeping its own copy of the truth.

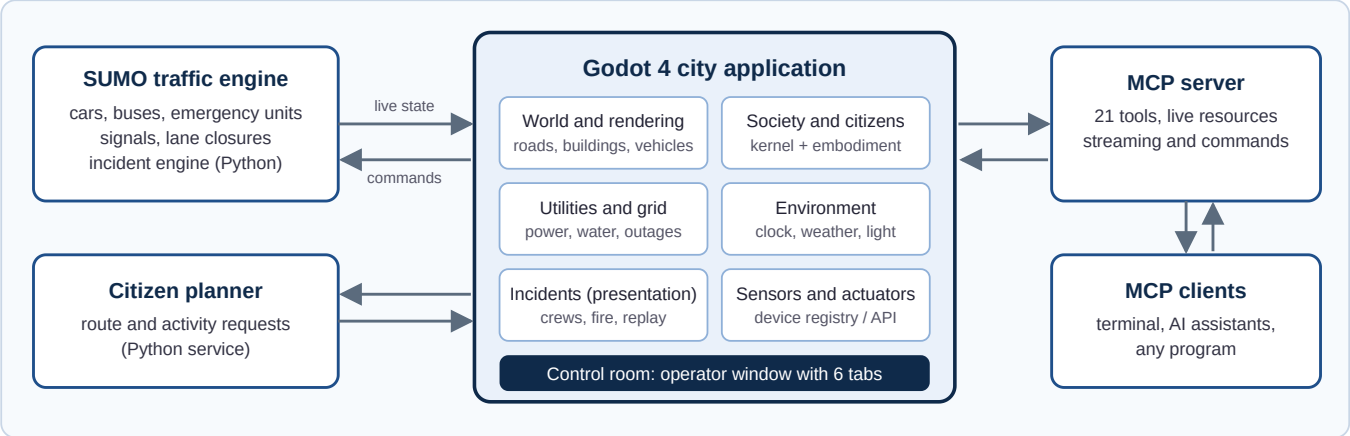
2. City plan and zones

The city is a regular grid of **8 × 8 districts** (64 city blocks) joined by 81 junctions and 144 road connections. The original 4 × 4 downtown core was extended to the full grid, and each block is zoned so that the city has a believable mix of uses:

Zone	Role in the simulation
Downtown	Skyscrapers, hotels and commerce; the heaviest power and water demand and the busiest streets.
Residential	Apartments and houses where the citizens' households live; lighting at night, evening and morning traffic.
Commercial and industrial	Shops, workplaces and service buildings that employ citizens and generate work orders.
Parks	Six parks with trees, benches and irrigation valves; the quietest and greenest areas.
Public services	Seven hospitals, the police building and emergency stations with vehicle bays, six water-reservoir compounds and two renewable energy hubs.

The 224 buildings are real 3D models with lit windows at night. Streets carry realistic furniture (benches, bins, vending machines, payphones, hydrants, parking meters, coffee carts), placed along kerbs by rules rather than at random. **49 signal-controlled intersections** regulate the interior of the network. Underneath sit two utility networks: a power grid (two renewable sources, a redundant ring of overhead lines, three switch cabinets) with a 42,000 kWh reserve, and a water system of six reservoirs that supply every block from its nearest reservoir. Each block's demand is computed from the buildings it contains, so a hospital block costs more than a residential one.

3. Architecture



The application is made of cooperating parts, each with a single responsibility, and every connection is a simple local message link (UDP or TCP on the same machine):

- **SUMO owns the road.** The position and speed of every vehicle, every traffic signal and every lane closure live in SUMO. A Python bridge runs SUMO continuously and publishes the whole traffic state to the city several times a second. Godot never moves a road vehicle on its own, so what you see is exactly what was simulated, and a visible collision is a simulated collision.
- **Godot owns the world.** It draws the city, runs the environment (clock, weather, lighting), the utility networks, the citizens' bodies, the incident scenes and the control room. Operator commands for traffic go back to SUMO over a small control channel.
- **The society kernel owns the citizens' lives.** Identity, money, jobs, households and daily agendas are kept separate from the visible 3D models. A model on screen is only the "body" of a citizen whose life exists whether or not they are drawn.
- **The incident engine owns emergencies.** It runs inside the traffic bridge, because a crash or a lane closure must physically affect SUMO traffic. Godot then presents the scene (wrecks, crews, fire and water effects).
- **The device interface owns access.** Sensors and actuators are read and driven by a single registry inside Godot, exposed through an MCP server. The control-room Sensors tab and external clients see the same devices and the same numbers.

> **Why this split.** Earlier versions let the presentation layer invent state (cars that vanished, citizens who got jobs because they stood next to a prop). Giving every fact one owner removed that whole class of inconsistency and makes each part testable on its own.

4. Simulation model

Traffic

Around 200 vehicles circulate in SUMO as a closed fleet that enters at the city edge and reroutes legally forever, with no teleporting. About 68 of the nearest are drawn as detailed lane-sized 3D vehicles of typed classes (car, taxi, bus, police, ambulance, fire, tow, utility). Vehicle dimensions in SUMO match the rendered models, signals follow SUMO's phases, and weather scales driver behaviour (rain and storm slow traffic).

Society

The population is deterministic and reproducible: 200 citizens in 80 households with jobs at 13 institutions and a graph of about 440 relationships. Each citizen follows a daily agenda (sleep, commute, work or school, meals, errands and leisure, return home) driven by the simulated clock. Institutions issue **work orders** (for example inspect a meter, service a hydrant, empty a bin) and a suitable employed citizen is assigned, walks there, performs the work, and the result changes wages, fatigue, budgets and asset condition. Needs such as thirst, hunger and energy send people to fountains, vending machines, benches and coffee carts, and they throw their rubbish in a bin afterwards. Up to 32 citizens are shown in detail at once; the rest continue as records.

Bodies and movement

Visible citizens use realistic Microsoft Rocketbox avatars and recorded motion-capture clips (sitting, crouching, working, phoning, drinking, reading). Instead of posing limbs by code, each person is placed so that the recorded hands land on the prop. They walk on a collision-checked sidewalk graph with lane discipline, giving way and recovery from blockages.

Environment and utilities

A continuous day/night clock (adjustable pace), five weather states, adaptive street lighting and lit building windows set the mood. Power and water drain in simulated time according to block demand; an operator or an automatic event can cut a service zone, and the affected blocks, meters and lights respond.

5. Sensors and actuators

All devices are real objects placed in the city (spatially checked to avoid roads and buildings) and are registered once, with a stable id such as *AIR-01* or *SWT-02*. There are **566 devices in 18 families**:

Group	Devices	What they report or do
Traffic and cameras	12 road radars, 16 street cameras	Vehicle count, mean speed, occupancy; live camera views
Environment	8 air, 4 noise, 4 weather, 4 flood-level	PM2.5/PM10/NO_2, sound level, wind, rain, temperature, water depth
Utilities	16 power and 16 water meters, 8 telemetry nodes, 6 reservoirs, 1 grid, 16 service zones	Demand, voltage, pressure, stored water and energy, supplied or cut
Waste	96 street cans and 44 green containers, each with a weight sensor	Weight, fill level and items deposited; rises when citizens throw rubbish in, resets when a truck empties it
Actuators	49 signals, 3 power switches, 5 water valves, 256 street lights, 4 irrigation valves	Open/close, on/off/auto/dim, phase changes; each command reports the readings before and after

The readings are **derived from the live city** rather than random: a traffic radar counts the simulated vehicles near it, an air monitor responds to nearby exhaust and rain, a bin weighs what citizens actually deposited. Commands go through the same code paths as the control-room buttons, so a command from an external client and a click by the operator are indistinguishable.

6. How scenarios are played

A scenario is a combination of conditions the operator sets and events that unfold on their own. The operator chooses the time of day, the weather, the simulation speed (1–8×) and the clock pace, pauses and resumes, and picks a camera or follows a random citizen or car. Then events play out:

- **Incidents.** Eleven types can be triggered from the Incidents tab or by schedule: rear-end collision, red-light T-bone, unsafe overtake, secondary collision, breakdown, vehicle fire, lost load, building fire, water-main break, signal failure and garbage collection (called automatically at night). The incident engine choreographs the event in SUMO so that lanes close, nearby junctions go under police control and cars reroute or queue.
- **Emergency response.** Police, ambulance, fire, tow, utility and sanitation units move through a lifecycle (stationed, dispatched, responding with lights, on scene, returning). Crews leave their vehicles, secure the scene, treat people or fight the fire, and tow units clear the wreck afterwards. Every incident can be recorded and replayed.
- **Infrastructure events.** Cutting power or water to a zone, opening a switch, closing a valve or storm flooding shows up immediately in meters, reservoirs, lights and the Sensors tab.
- **Daily life.** Without any operator input the city lives on its own: people commute, work, queue for drinks, sit on benches and go home, and the night lighting and refuse collection follow the clock.

Scenarios can also be **scripted from outside**: a client connects through MCP, sets the weather, cuts a zone, watches the affected sensors stream live and restores service, with no extra code inside the city.

7. The MCP interface

The Model Context Protocol is the standard way for AI assistants and programs to use tools. The city exposes **21 functions**: read one or many sensors, summaries and bin reports, stream a sensor for a given duration, list the actuators and their allowed commands, and command signals, switches, valves, lights, zones, weather, clock and speed. The reading functions are live by themselves: called by a client that can show progress, they stream an update every second until the call is cancelled, and a single call returns the whole series (or one reading with *once*). MCP resource subscriptions can also push a notice whenever a value changes. The terminal client shipped with the project adds nothing to these calls (it sends what is typed and prints what comes back), so every MCP client behaves exactly as the terminal does; any client finds the city automatically through a well-known port.

8. Technology stack

Layer	Choice and why
3D engine and application	**Godot 4.7** (Vulkan, Forward+): free, scriptable, fast to iterate, a single process for world, control room and device interface.
Traffic	**Eclipse SUMO 1.25** with the TraCI interface: a validated microscopic traffic simulator with signals, lane control and routing.
Glue and engines	**Python 3.14**: traffic bridge, incident and emergency engine, citizen planner and the MCP server; local UDP/TCP messages in JSON.
Gameplay logic	**GDScript** for the city systems (society kernel, utilities, environment, device registry, control room).
Assets	Poly Pizza, Poly Haven, Kenney and Quaternius models (CC0/CC-BY); **Microsoft Rocketbox** avatars and motion capture (MIT).
Interface to the outside	**MCP** over standard input/output (JSON-RPC 2.0), with progress notifications and resource subscriptions.
Packaging	Self-contained: a runtime folder bundles Godot, SUMO and Python, launched by two shell scripts.

9. Quality and verification

The project has about 90 automated Godot test scripts plus Python tests for SUMO behaviour, the incident engine and the MCP interface. They check, among other things, that every prop is grounded and correctly sized, that citizens' hands are on the props they use, that nobody walks through obstacles, that every sensor family exists and answers, that actuators really change the city, and that the whole chain from a terminal command to the running simulation works end to end.

Known limits. Sensors are modelled from city state rather than measured by hardware; the city is one fixed map; valve actuators change state and indicators but do not yet restrict flow in the network. Natural next steps are more scenarios (flooding and power-failure chains), more citizen routines and recorded scenario playlists.